

// javascript

```
db.users.insertMany([
  {
    _id: ObjectId(),
    name: "Tarun Patidar",           // string
    age: 22,                         // number
    salary: 45500.75,                // double
    isActive: true,                  // boolean
    phone: "9876543210",             // phone number
    aadhaar: "123456789012",         // aadhaar number
    password: "Tarun@1234Abcd$X",    // password
    dob: new Date("2004-08-15"),     // date
    middleName: null,                // null
    tempField: undefined,            // undefined

    address: {                       // embedded document
      city: "Indore",
      state: "Madhya Pradesh",
      pincode: 452010
    },

    skills: ["C++", "MongoDB", "Node.js"], // array
```

```
education: [                                     // array of embedded documents
  { level: "10th", year: 2018, percentage: 82.4 },
  { level: "12th", year: 2020, percentage: 78.6 }
],

extraInfo: { hobby: "Cricket" },                // schemaless field
profilePic: BinData(0, "U29tZURhdGE="), // binary data
createdAt: new Date()
},

{
  _id: ObjectId(),
  name: "Priya Verma",
  age: 24,
  salary: 52000.50,
  isActive: false,
  phone: "8123456789",
  aadhaar: "987654321098",
  password: "Priya#5678Xyzm@Q",
  dob: new Date("2002-03-20"),
  middleName: null,
```



// JavaScript

```
insertOne({  
  name: "Rishi",  
  address: {  
    city: "Bhopal",  
    state: "Madhya Pradesh",  
    pincode: 462001  
  },  
  skills: ["Java", "SQL"],  
  education: [  
    { level: "BTech", college: "LNCT", cgpa: 8.1 }  
  ],  
  extraInfo: { favoriteColor: "Blue", expectedCTC: 600000 },  
  createdAt: new Date()  
})
```

/c/0a30b0e1-ec10-03ee-929e-0700e12b4e41

Default behavior

JavaScript

```
{ ordered: true }
```

Meaning

- Documents **sequence me insert** honge
- Agar beech me **error** aa gaya, to **uske baad wale documents** insert nahi honge

Example

JavaScript

```
db.users.insertMany(  
  [  
    { _id: 1, name: "Tarun" },  
    { _id: 1, name: "Priya" }, // duplicate _id -> error  
    { _id: 3, name: "Rahul" }  
  ],  
  { ordered: true }  
)
```



2) Unordered Insert

Syntax

```
<> JavaScript  
  
{ ordered: false }
```

Meaning

- Agar kisi ek document me error aaye, tab bhi **baaki valid documents** insert hote rahenge

Example

```
<> JavaScript  
  
db.users.insertMany(  
  [  
    { _id: 1, name: "Tarun" },  
    { _id: 1, name: "Priya" }, // duplicate _id -> error  
    { _id: 3, name: "Rahul" }  
  ],  
  { ordered: false }  
)
```



MongoDB BSON Data Types

[Upgr](#)

Data Type	BSON Type Number	Size	Use	
String	2	Variable	Name, Email, Address	
Int32	16	4 bytes	Age, Marks	
Int64 (Long)	18	8 bytes	Large IDs, Big Numbers	
Double	1	8 bytes	Decimal approximation	
Decimal128	19	16 bytes	Money, Salary	
Boolean	8	1 byte	true / false	
Date	9	8 bytes	DOB, CreatedAt	
Timestamp	17	8 bytes	Replication, Internal	
ObjectId	7	12 bytes	Default <code>_id</code>	

MongoShell command

ObjectId	7	12 bytes	Default <code>_id</code>	
Array	4	Variable	Skills, Subjects	
Object (Document)	3	Variable	Nested Documents	
Null	10	-	Null values	
Binary	5	Variable	Files, Images	
Regular Expression	11	Variable	Regex	
JavaScript	13	Variable	JS Code (Rare)	
MinKey	-1	-	Internal	
MaxKey	127	-	Internal	

SQL vs MongoDB	
SQL	MongoDB
INT	Int32
BIGINT	Int64 (Long)
FLOAT	Double
DECIMAL	Decimal128
VARCHAR	String
CHAR	String
TEXT	String
BOOLEAN	Boolean
DATE	

MongoShell command

▼	BOOLEAN	Boolean
	DATE	Date
	TIMESTAMP	Timestamp / Date
	BLOB	Binary
	Table	Collection
	Row	Document
	Column	Field

Safe version (pehle copy check karo, baad me old db delete karo)

JavaScript

```
use companyDB
```

```
db.getCollectionNames().forEach(function(collName) {  
  const docs = db[collName].find().toArray();  
  if (docs.length > 0) {  
    db.getSiblingDB("corporateDB")[collName].insertMany(docs);  
  }  
})
```

Phir check karo:

⌘ JavaScript

```
db.orders.renameCollection("salesOrders")
```

5) payments → salaryPayments

⌘ JavaScript

```
db.payments.renameCollection("salaryPayments")
```

6) courses → trainingPrograms

⌘ JavaScript

```
db.courses.renameCollection("trainingPrograms")
```

Haan 👍

MongoDB shell me `updateOne()` ka syntax ye hota hai:

Syntax

⌘ JavaScript

```
db.collectionName.updateOne(  
  { condition },  
  { $set: { field: value } }  
)
```

1) employee ka city update karna

Agar employee ka naam **Tarun Patidar** hai aur uska city **Indore** karna hai:

</> JavaScript

```
db.employees.updateOne(  
  { name: "Tarun Patidar" },  
  { $set: { city: "Indore" } }  
)
```

2) student ki fees update karna

</> JavaScript

```
db.students.updateOne(  
  { name: "Rahul Sharma" },  
  { $set: { fees: 75000 } }  
)
```



Multiple fields ek saath update

JavaScript

```
db.employees.updateOne(
  { name: "Tarun Patidar" },
  { $set: { city: "Pune", salary: 95000, department: "Development" } }
)
```

Agar value increase karni ho (\$inc)

salary me 5000 add

JavaScript

```
db.employees.updateOne(
  { name: "Tarun Patidar" },
  { $inc: { salary: 5000 } }
)
```



Agar field remove karni ho (\$unset)

⌘ JavaScript

```
db.employees.updateOne(  
  { name: "Tarun Patidar" },  
  { $unset: { bonus: "" } }  
)
```

Result check karna

⌘ JavaScript

```
db.employees.find({ name: "Tarun Patidar" })
```

updateOne() vs updateMany()

- `updateOne()` → sirf 1 matching document update karega
- `updateMany()` → jitne bhi matching documents honge sab update karega

Tumhare database ke hisaab se examples

1) Indore ke sab students ka department update

JavaScript

```
db.students.updateMany(  
  { city: "Indore" },  
  { $set: { department: "CSE" } }  
)
```


es



com/c/6a50b8ef-ec10-83ee-929e-0780ef2b4e4f

8) Sab customers se likes field hata do

</> JavaScript

```
db.customers.updateMany(  
  {},  
  { $unset: { likes: "" } }  
)
```

Multiple fields ek saath update

9) Pune ke employees ka city aur department dono update

</> JavaScript

```
db.employees.updateMany(  
  { city: "Pune" },  
  { $set: { city: "Pune Maharashtra", department: "Software" } }  
)
```



Condition ke saath range update

10) Jinki salary 100000 se kam hai unka bonus increase

`</>` JavaScript

```
db.employees.updateMany(  
  { salary: { $lt: 100000 } },  
  { $inc: { bonus: 3000 } }  
)
```

Result check karna

`</>` JavaScript

```
db.employees.find({ department: "IT" })
```

ya



fdff

1. `deleteOne()` — ek document delete

 JavaScript

```
db.firstCollection.deleteOne({  
  name: "Rahul"  
})
```

 `name: "Rahul"` wala ek matching document delete hoga.

2. `deleteMany()` — multiple documents delete

 JavaScript

```
db.firstCollection.deleteMany({  
  city: "Indore"  
})
```

 `city = "Indore"` wale saare documents delete honge.



3. `findOneAndDelete()` — find + delete

</> JavaScript

```
db.firstCollection.findOneAndDelete({  
  name: "Rahul"  
})
```

➡ Ek matching document delete hoga aur **deleted document return** hoga.

4. `deleteById()` — Mongo Shell me direct method nahi hai ❌

Mongo Shell me `_id` se delete karne ke liye:

</> JavaScript

```
db.firstCollection.deleteOne({  
  _id: ObjectId("66c123abc456789012345678")  
})
```



1. `$and` ke saath `deleteMany()`

Suppose delete karna hai:

age 23 se 55 ke beech **AND** city = nalkheda

`</>` JavaScript

```
db.firstCollection.deleteMany({
  $and: [
    { age: { $gte: 23 } },
    { age: { $lte: 55 } },
    { city: "nalkheda" }
  ]
})
```

Lekin `$and` ki zarurat nahi bhi hai

MongoDB me ye same hai:

</> JavaScript

```
db.firstCollection.deleteMany({
  age: {
    $gte: 23,
    $lte: 55
  },
  city: "nalkheda"
})
```

Top-level fields automatically **AND** hote hain.

2. \$or ke saath deleteMany()

city Indore **OR** Bhopal wale documents delete karo.

</> JavaScript

```
db.firstCollection.deleteMany({
  $or: [
    { city: "Indore" },
    { city: "Bhopal" }
  ]
})
```

3. \$in ke saath deleteMany()

City Indore, Bhopal ya Ujjain me se koi bhi ho.

</> JavaScript

```
db.firstCollection.deleteMany({  
  city: {  
    $in: ["Indore", "Bhopal", "Ujjain"]  
  }  
})
```

Ye basically:

```
city = Indore  
OR  
city = Bhopal  
OR  
city = Ujjain
```



5. \$and + \$or

Suppose:

age 23+ **AND** city Indore **OR** Bhopal

</> JavaScript

```
db.firstCollection.deleteMany({
  $and: [
    { age: { $gte: 23 } },
    {
      $or: [
        { city: "Indore" },
        { city: "Bhopal" }
      ]
    }
  ]
})
```

`deleteOne()` vs `deleteMany()`

`deleteOne()`

- Sirf 1 document delete

`deleteMany()`

- Saare matched documents delete

2 important examples

1) One employee delete

JavaScript

```
db.employees.deleteOne({ name: "Tarun Patidar" })
```



om/c/6a50b8ef-ec10-83ee-929e-0780ef2b4e4f

</> JavaScript

```
db.collectionName.updateOne(  
  { condition },  
  { $rename: { "oldFieldName": "newFieldName" } }  
)
```

Example 1: single document me field rename

mobile field ko phone banana:

</> JavaScript

```
db.students.updateOne(  
  { name: "Rahul Sharma" },  
  { $rename: { "mobile": "phone" } }  
)
```

Meaning: Rahul Sharma wale document me mobile field ka naam phone ho jayega.

Example 2: sab documents me field rename

Agar poori collection me `mobile` ko `phone` banana ho:

JavaScript

```
db.students.updateMany(  
  {},  
  { $rename: { "mobile": "phone" } }  
)
```

Tumhare database ke useful examples

employees me `post` → `designation`

JavaScript

```
db.employees.updateMany(  
  {},  
  { $rename: { "post": "designation" } }  
)
```



'6a50b8ef-ec10-83ee-929e-0780ef2b4e4f'

1) Update Array

Array me new value add → `$push`

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $push: { tags: "backend" } }  
)
```

Meaning: `tags` array me `"backend"` add hoga.

Array me multiple values add → `$push + $each`

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $push: { tags: { $each: ["nodejs", "express"] } }  
)
```



c/0a30b0e1-ec10-03ee-929e-0700e12b4e41

Embedded document me naya object add

JavaScript

```
db.socialPosts.updateOne(
  { _id: 1 },
  {
    $push: {
      comments: { user: "Tarun", msg: "Excellent" }
    }
  }
)
```

Embedded document remove karna

JavaScript

```
db.socialPosts.updateOne(
  { _id: 1 },
  { $pull: { comments: { user: "Rahul" } } }
)
```



2) Embedded Document Update

Tumhare `socialPosts` me `comments` embedded documents ka array hai:

JavaScript

```
comments: [  
  { user: "Rahul", msg: "Nice" },  
  { user: "Priya", msg: "Good" }  
]
```

Embedded document ka field update

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $set: { "comments.0.msg": "Very Nice" } }  
)
```



:/6a50b8ef-ec10-83ee-929e-0780ef2b4e4f

Array se value remove → \$pull

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $pull: { tags: "mongodb" } }  
)
```

Meaning: tags array se "mongodb" remove ho jayega.

Array me ek specific index update

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $set: { "tags.0": "database" } }  
)
```



Meaning: tags array ka first element update hoga.

3 Most Important Examples

1) Tag add

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $push: { tags: "interview" } }  
)
```

2) First comment message update

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $set: { "comments.0.msg": "Awesome" } }  
)
```



717/5/0420001-0010-0000-0000-0700012B7C71

Example: first tag remove

</> JavaScript

```
db.socialPosts.updateOne(
  { _id: 1 },
  { $pop: { tags: -1 } }
)
```

2) Embedded document array se remove → \$pull

Agar embedded documents ka array ho, jaise:

</> JavaScript

```
comments: [
  { user: "Rahul", msg: "Nice" },
  { user: "Priya", msg: "Good" }
]
```



`$pop` array ke start ya end se element remove karta hai.

- `1` → last element remove
- `-1` → first element remove

Syntax

JavaScript

```
db.collectionName.updateOne(  
  { condition },  
  { $pop: { arrayField: 1 } } // last remove  
)
```

Example: last tag remove

JavaScript

```
db.socialPosts.updateOne(  
  { _id: 1 },  
  { $pop: { tags: 1 } }  
)
```



Important difference

`$pop`

- Sirf **first/last element** remove karta hai
- Specific value/object target nahi karta

`$pull`

- **Matching value ya object** remove karta hai
- Arrays aur embedded document arrays dono me useful

Important difference

`findOne()`

- single document return karta hai

`find()`

- cursor return karta hai / multiple documents de sakta hai

Most important examples

1) Ek user find karna

 JavaScript

```
db.users.findOne({ phone: "9876543210" })
```



2) Indore ke saare users

</> JavaScript

```
db.users.find({ "address.city": "Indore" })
```

3) Age 22 wale users

</> JavaScript

```
db.users.find({ age: 22 })
```

4) Active users

</> JavaScript

```
db.users.find({ isActive: true })
```

Nested / Embedded field read

Embedded document ke field ko **dot notation** se access karte hain:

⌞ JavaScript

```
db.users.find({ "address.city": "Bhopal" })
```

Array field read

Agar array me "MongoDB" skill ho:

⌞ JavaScript

```
db.users.find({ skills: "MongoDB" })
```

MongoShell command